

```
template<typename T>
struct FenwickTree
{
    int n,bit;
    vector<T>BIT;
    FenwickTree(int _n):BIT(_n+1),n(_n)
    {
        bit=1;
        while((bit<<1)<=n)bit<<=1;
    }

    inline int lb(int x){return -x&x;}

    inline void add(int x,const T &v)
    {
        for(int i=x;i<=n;i+=lb(i))
        {
            BIT[i]=BIT[i]+v;
        }
    }

    inline T sum(int x)
    {
        T res=0;
        for(int i=x;i>0;i-=lb(i))
        {
            res=res+BIT[i];
        }
        return res;
    }

    inline T rangeSum(int l,int r)
    {
        return sum(r)-sum(l-1);
    }

    inline T select(const T& k) //返回第一个前缀和大于等于k的下标
    {
        int idx=0;
        T cur{};
        for(int i=bit;i>=1)
        {
            if(idx+bit>n||cur+BIT[idx+bit]>k)continue;
            idx+=bit;
            cur=cur+BIT[idx];
        }
        return idx;
    }
};
```

线段树

```
struct Node
{
    int sum,maxv,minv;
    int lazy;
}segTree[maxn<<2];

void build(int x,int l,int r)
{
    segTree[x].lazy=0;
    if(l==r)
    {
        segTree[x]={A[l],A[l],A[l],0};
        return;
    }
    int mid=(l+r)>>1;
    int ls=x<<1,rs=x<<1|1;
    build(ls,l,mid);
    build(rs,mid+1,r);
    segTree[x].sum=segTree[ls].sum+segTree[rs].sum;
    segTree[x].maxv=max(segTree[ls].maxv,segTree[rs].maxv);
    segTree[x].minv=min(segTree[ls].minv,segTree[rs].minv);
}

void pushDown(int x,int l,int r)
{
    if(segTree[x].lazy==0)return;
    int lz=segTree[x].lazy;
    int mid=(l+r)>>1;
    int ls=x<<1,rs=x<<1|1;
    segTree[ls].sum+=(mid-l+1)*lz;
    segTree[ls].maxv+=lz;
    segTree[ls].minv+=lz;
    segTree[ls].lazy+=lz;
    segTree[rs].sum+=(r-mid)*lz;
    segTree[rs].maxv+=lz;
    segTree[rs].minv+=lz;
    segTree[rs].lazy+=lz;
    segTree[x].lazy=0;
}

void rangeAdd(int ql,int qr,int val,int x,int l,int r)
{
    if(ql<=l&&r<=qr)
    {
        segTree[x].sum+=(r-l+1)*val;
        segTree[x].maxv+=val;
        segTree[x].minv+=val;
        segTree[x].lazy+=val;
        return;
    }
}
```

```

    }
    pushDown(x,l,r);
    int mid=(l+r)>>1;
    int ls=x<<1,rs=x<<1|1;
    if(ql<=mid)rangeAdd(ql,qr,val,ls,l,mid);
    if(qr>=mid+1)rangeAdd(ql,qr,val,rs,mid+1,r);
    segTree[x].sum=segTree[ls].sum+segTree[rs].sum;
    segTree[x].maxv=max(segTree[ls].maxv,segTree[rs].maxv);
    segTree[x].minv=min(segTree[ls].minv,segTree[rs].minv);
}

Node rangeQuery(int ql,int qr,int x,int l,int r)
{
    if(ql<=l&&r<=qr)return segTree[x];
    pushDown(x,l,r);
    Node res={0,-MAX,MAX,0};
    int mid=(l+r)>>1;
    int ls=x<<1,rs=x<<1|1;
    if(ql<=mid)
    {
        Node left=rangeQuery(ql,qr,ls,l,mid);
        res.sum+=left.sum;
        res.maxv=max(res.maxv,left.maxv);
        res.minv=min(res.minv,left.minv);
    }
    if(qr>=mid+1)
    {
        Node right=rangeQuery(ql,qr,rs,mid+1,r);
        res.sum+=right.sum;
        res.maxv=max(res.maxv,right.maxv);
        res.minv=min(res.minv,right.minv);
    }
    return res;
}

```

主席树

```

struct Node
{
    int ls,rs,v;
}str[maxn<<6];
int n,cnt,a[maxn],ver[maxn];

void update(int &o1,int o2,int l,int r,int pos,int v)
{
    str[o1=++cnt]=str[o2];
    if(l==r)
    {
        str[o1].v+=v;
        return;
    }
}

```

```

    int mid=(l+r)>>1;
    if(pos<=mid)update(str[o1].ls,str[o2].ls,l,mid,pos,v);
    else update(str[o1].rs,str[o2].rs,mid+1,r,pos,v);
    str[o1].v=str[str[o1].ls].v+str[str[o1].rs].v;
}
int search(int o1,int o2,int l,int r,int ql,int qr)
{
    if(ql<=l&&r<=qr)
    {
        return str[o2].v-str[o1].v;
    }
    int mid=(l+r)>>1,res=0;
    if(ql<=mid)res+=search(str[o1].ls,str[o2].ls,l,mid,ql,qr);
    if(qr>mid)res+=search(str[o1].rs,str[o2].rs,mid+1,r,ql,qr);
    return res;
}

void sol()
{
    int m;
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
        update(ver[i],ver[i-1],1,INF,a[i],a[i]);
    }
    cin>>m;
    while(m-->0)
    {
        int l,r;
        cin>>l>>r;//l-1版本和r版本之差的权值树
        int lst=0,ans=1;
        while(1)
        {
            int mob=search(ver[l-1],ver[r],1,INF,lst+1,ans);
            if(mob>0)
            {
                lst=ans;
                ans+=mob;
            }
            else break;
        }
        cout<<ans<<'\\n';
    }
}

```

RMQ

```

typedef int T;
struct RMQ
{

```

```

vector<int>lg;
vector<vector<T>>ST;
int N;

RMQ(int n,vector<T>&a):N(n)
{
    lg.assign(n+1,0);
    ST.assign(21,vector<T>(n+1));
    for(int i=1;i<=n;i++)ST[0][i]=a[i];
    for(int i=2;i<=N;i++)lg[i]=lg[i/2]+1;
    build();
}

inline T merge(T a,T b)
{
    return max(a,b);
}

void build()
{
    for(int i=1;i<20;i++)
    {
        int len=1<<i;
        for(int j=1;j+len-1<=N;j++)
        {
            ST[i][j]=merge(ST[i-1][j],ST[i-1][j+(1<<(i-1))]);
        }
    }
}

inline T search(int l,int r)
{
    int len=r-l+1;
    int t=lg[len];
    return merge(ST[t][l],ST[t][r-(1<<t)+1]);
}
};

```

LCA1

```

vector<vector<int>>edge(maxn),f(maxn,vector<int>(21));
vector<int>h(maxn);

void dfs(int x,int fa)
{
    f[x][0]=fa;
    h[x]=h[fa]+1;
    for(int i=1;i<21;i++)
    {
        f[x][i]=f[f[x][i-1]][i-1];
    }
}

```

```

    for(auto v:edge[x])
    {
        if(v==fa)continue;
        dfs(v,x);
    }
}

int LCA(int a,int b)
{
    if(h[a]>h[b])swap(a,b);
    int gap=h[b]-h[a];
    int w=0;
    while(gap)
    {
        if(gap&1)b=f[b][w];
        gap>>=1;
        w++;
    }
    if(a==b)return a;
    for(int i=20;i>=0;i--)
    {
        if(f[a][i]==f[b][i])continue;
        a=f[a][i];
        b=f[b][i];
    }
    return f[a][0];
}

```

LCA2

```

vector<vector<int>>>edge(maxn);
vector<vector<pair<int,int>>>>query(maxn);
bitset<maxn>vis;
vector<int>F(maxn),ans(maxn);

int Find(int x)
{
    if(x==F[x])return x;
    return F[x]=Find(F[x]);
}

void merge(int x,int fa)
{
    F[Find(x)]=Find(fa);
}

void Tarjan_DFS(int x,int fa=0)
{
    vis[x]=1;
    for(auto [y,id]:query[x])
    {
        if(!vis[y])continue;
    }
}

```

```

        ans[id]=Find(y);
    }
    for(auto v:edge[x])
    {
        if(v==fa)continue;
        Tarjan_DFS(v,x);
    }
    merge(x,fa);
}

```

Mo

```

struct Q
{
    int l,r,id;
};
vector<Q>q;
vector<int>ans(maxn);
int B;

bool cmp(const Q &x,const Q &y)
{
    int bx=x.l/B,by=y.l/B;
    if(bx!=by)return bx<by;
    return (bx&1)?(x.r>y.r):(x.r<y.r);
}

void add(int pos,int &cur)
{
}

void del(int pos,int &cur)
{
}

void Mo()
{
    sort(q.begin(),q.end(),cmp);
    int curL=1,curR=0,cur=0;
    for(auto [l,r,id]:q)
    {
        while(curL>l)add(--curL,cur);
        while(curR<r)add(++curR,cur);
        while(curL<l)del(++curL,cur);
        while(curR>r)del(--curR,cur);
        ans[id]=cur;
    }
}

```

Dinic

```

struct DinicMaxFlow
{
    struct Edge
    {
        int to, cap, rev;
    };

    int N, S, T;
    const int INF=2e18;
    vector<vector<Edge>>e;
    vector<int>cur, level;

    DinicMaxFlow(int n, int s, int t):N(n), S(s), T(t)
    {
        e.assign(n+1, {});
        cur.assign(n+1, 0);
        level.assign(n+1, 0);
    }

    void add_edge(int u, int v, int cap)
    {
        e[u].emplace_back(v, cap, e[v].size());
        e[v].emplace_back(u, 0, e[u].size()-1);
    }

    bool BFS()
    {
        fill(begin(level), end(level), 0); //分层图
        fill(begin(cur), end(cur), 0); //重置当前弧
        queue<int>q;
        level[S]=1;
        q.emplace(S);
        while(!q.empty())
        {
            int u=q.front();
            q.pop();
            for(auto [v, c, r]:e[u])
            {
                if(level[v]||c==0)continue;
                level[v]=level[u]+1;
                q.emplace(v);
            }
        }
        return level[T];
    }

    int DFS(int u, int f)
    {
        if(u==T||f==0)return f;
        int ret=0;
        //多路增广
    }
}

```



```

        for(int &i=cur[u];i<(int)e[u].size();i++)    //&i=cur[u],当前弧优化
        {
            auto &[v,c,r]=e[u][i];
            if(c==0||level[v]!=level[u]+1)continue;
            int tf=DFS(v,min(f-ret,c));
            if(tf==0)continue;
            c-=tf;
            e[v][r].cap+=tf;
            ret+=tf;
            if(f==ret)break;
        }
        return ret;
    }

    int MaxFlow()
    {
        int res=0;
        while(BFS())res+=DFS(S,INF);
        return res;
    }
};

void sol()
{
    int n,m,s,t;
    cin>>n>>m>>s>>t;
    DinicMaxFlow D(n,s,t);
    for(int i=1;i<=m;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        D.add_edge(u,v,w);
    }
    cout<<D.MaxFlow();
}

```

MCMF

```

struct MCMF
{
    struct Edge
    {
        int to,cap,cost,rev;
    };

    int N,S,T,MC,MF;
    const int INF=2e18;
    vector<vector<Edge>>>e;
    vector<int>cur,dist,vis;

    MCMF(int n,int s,int t):N(n),S(s),T(t),MC(0),MF(0)

```

```
{
    e.assign(n+1,vector<Edge>());
    cur.assign(n+1,0);
    dist.assign(n+1,0);
    vis.assign(n+1,0);
}

void add_edge(int u,int v,int cap,int cost)
{
    e[u].emplace_back(v,cap,cost,e[v].size());
    e[v].emplace_back(u,0,-cost,e[u].size()-1);
}

bool SPFA()
{
    fill(begin(dist),end(dist),INF);
    fill(begin(vis),end(vis),0);
    fill(begin(cur),end(cur),0);           //重置当前弧
    queue<int>q;
    dist[S]=0;
    vis[S]=1;
    q.emplace(S);
    while(!q.empty())
    {
        int u=q.front();
        q.pop();
        vis[u]=0;
        for(auto [v,c,m,r]:e[u])
        {
            if(c&&dist[u]+m<dist[v])
            {
                dist[v]=dist[u]+m;
                if(!vis[v])
                {
                    vis[v]=1;
                    q.emplace(v);
                }
            }
        }
    }
    return dist[T]!=INF;
}

int DFS(int u,int f)
{
    if(u==T||f==0)return f;
    vis[u]=1;           //防止出现0费用环
    int ret=0;
    for(int &i=cur[u];i<(int)e[u].size();i++)
    {
        auto &[v,c,m,r]=e[u][i];
        if(vis[v]||c==0||dist[v]!=dist[u]+m)continue;
        int tf=DFS(v,min(f-ret,c));
        if(tf==0)continue;
```

```

        c-=tf;
        e[v][r].cap+=tf;
        ret+=tf;
        MC+=tf*m;
        if(f==ret)break;
    }
    vis[u]=0;
    return ret;
}

void mcmf()
{
    while(SPFA())MF+=DFS(S,INF);
}
};

```

Johnson

```

bool SPFA(int s)
{
    h.assign(n+1,INF);
    h[s]=0;
    queue<int>q;
    vector<int>vis(n+1,0),cnt(n+1,0);
    q.emplace(s);
    vis[s]=1;
    while (!q.empty())
    {
        int u=q.front();
        q.pop();
        vis[u]=0;
        for(auto [v,w]:edge[u])
        {
            if(h[u]+w<h[v])
            {
                h[v]=h[u]+w;
                if(!vis[v])
                {
                    q.push(v);
                    vis[v]=1;
                    cnt[v]++;
                    if(cnt[v]>=n)return true;
                }
            }
        }
    }
    return false;
}

void Dijkstra(int s)
{

```

```

priority_queue<pair<int,int>>q;
q.emplace(0,s);
dist[s][s]=0;
while(!q.empty())
{
    auto[d,u]=q.top();
    q.pop();
    d=-d;
    if(d>dist[s][u])continue;
    for(auto [v,w]:edge[u])
    {
        if(dist[s][v]>d+w)
        {
            dist[s][v]=d+w;
            q.emplace(-dist[s][v],v);
        }
    }
}

void Johnson()
{
    for(int i=1;i<=n;i++)
    {
        edge[0].emplace_back(i,0);
    }
    if(SPFA(0))
    {
        cout<<-1;
        return;
    }
    dist.assign(n+1,vector<int>(n+1,INF));
    for(int u=1;u<=n;u++)
    {
        for(auto &[v,w]:edge[u])
        {
            w=w+h[u]-h[v];      //dist[v]<=dist[u]+w边权转化
        }
    }
    for(int i=1;i<=n;i++)Dijkstra(i);
    for(int i=1;i<=n;i++)
    {
        int res=0;
        for(int j=1;j<=n;j++)
        {
            if(dist[i][j]!=INF)dist[i][j]=dist[i][j]-h[i]+h[j];
            res+=j*dist[i][j];
        }
        cout<<res<<'\\n';
    }
}

```

Geometry

```

const long double eps=1e-10;
const long double PI=acos(-1.0);

typedef double T;

int sgn(T x)
{
    if(fabs(x)<eps)return 0;
    return x>0?1:-1;
}

struct PT
{
    T x,y;
    PT(){}
    PT(T _x,T _y):x(_x),y(_y){}

    PT operator+(const PT& b)const{return PT(x+b.x,y+b.y);}
    PT operator-(const PT& b)const{return PT(x-b.x,y-b.y);}
    PT operator*(const T& n)const{return PT(x*n,y*n);}
    PT operator/(const T& n)const{return PT(x/n,y/n);}
    bool operator==(const PT& b)const{return sgn(x-b.x)==0&&sgn(y-b.y)==0;}
    bool operator!=(const PT& b)const{return !(*this==b);}
    bool operator<(const PT& b)const{return (sgn(x-b.x)==0?(sgn(y-b.y)<0):(sgn(x-
b.x)<0));}
};

ostream& operator<<(ostream& os,PT a){return os<<'('<<a.x<<','<<a.y<<')';}

struct Line
{
    PT s,t;
    Line(){}
    Line(PT _s,PT _t):s(_s),t(_t){}
};

T dot(PT a,PT b){return a.x*b.x+a.y*b.y;} //点积
T cross(PT a,PT b){return a.x*b.y-a.y*b.x;} //叉积
T vabs(PT a){return sqrt(dot(a,a));} //模长
T dist(PT a,PT b){return vabs(a-b);} //两点距离

PT rot(PT a,T rad) //逆时针旋转rad角
{
    return PT(a.x*cos(rad)-a.y*sin(rad),a.x*sin(rad)+a.y*cos(rad));
}

PT normal(PT a) //逆时针的单位法向量
{
    if(vabs(a)<eps)return PT(0,0);
    return PT(-a.y,a.x)/vabs(a);
}

```

```

}

T angle(PT a,PT b) //向量夹角
{
    return abs(atan2(cross(a,b),dot(a,b)));
}

T orientedAngle(PT A,PT B,PT C) //定向角(逆时针)
{
    T ang=atan2(cross(B-A,C-A),dot(B-A,C-A));
    return ang+(ang<0)*2*PI;
}

T orient(PT A,PT B,PT C) //ABC的有向面积,逆时针为正(+)
{
    return cross(B-A,C-A)/2.0;
}

bool is_inAngle(PT A,PT B,PT C,PT p) //判断p点是否在射线AB和AC之间(含边界)
{
    T s=sgn(orient(A,B,C));
    if(s<0)swap(B,C);
    else if(s==0)
    {
        if(sgn(dot(B-A,C-A))>0) //特判ABC共线,同向则p也同向才满足
        {
            return sgn(cross(B-A,p-A))==0&&sgn(dot(B-A,p-A))>=0;
        }
        else return sgn(cross(B-A,p-A))>=0; //若反向则全满足
    }
    return sgn(orient(A,B,p))>=0&&sgn(orient(A,C,p))<=0;
}

bool is_onLine(PT p,Line l) //点p是否在直线ab上
{
    auto [s,t]=l;
    return sgn(cross(p-s,p-t))==0;
}

bool is_lineInter(Line l1,Line l2) //直线l1和l2是否相交
{
    auto [s1,t1]=l1;
    auto [s2,t2]=l2;
    if(sgn(cross(t1-s1,t2-s2))==0)
    {
        if(is_onLine(s1,l2))return 1;
        return 0;
    }
    return 1;
}

bool is_onSeg(PT p,Line l) //点p是否在线段ab上(含端点)
{
    auto [s,t]=l;

```

```

    return sgn(cross(p-s,p-t))==0&&sgn(dot(p-s,p-t))<=0;
}
bool is_onTwoSide(PT A,PT B,Line l)          //点AB是否在直线l两侧(不包含点在直线上)
{
    auto [s,t]=l;
    return sgn(cross(A-s,t-s))*sgn(cross(B-s,t-s))<0;
}
bool is_segInter(Line l1,Line l2)           //线段l1和l2是否相交(含端点)
{
    auto [s1,t1]=l1;
    auto [s2,t2]=l2;
    if(is_onSeg(s1,l2)||is_onSeg(t1,l2)||is_onSeg(s2,l1)||is_onSeg(t2,l1))
    {
        return 1;
    }
    return is_onTwoSide(s1,t1,l2)&&is_onTwoSide(s2,t2,l1);
}

bool polar_cmp(PT a,PT b)                  //极角排序(以原点为基准)
{
    int f1=(a.y>0||a.y==0&&a.x>0)?0:1;
    int f2=(b.y>0||b.y==0&&b.x>0)?0:1;
    if(f1!=f2)return f1<f2;
    int s=sgn(cross(a,b));
    if(s!=0)return s>0;
    return dot(a,a)<dot(b,b);//这里用平方判长度
}

bool is_onRight(PT p,Line l)
{
    auto [s,t]=l;
    return sgn(cross(t-s,p-s))>=0;
}
vector<PT>getConvex(vector<PT>a)           //获得凸包
{
    if(a.size()<=2)return a;
    PT st=a[0];
    for(auto p:a)
    {
        if(p.x<st.x||p.x==st.x&&p.y<st.y)
        {
            st=p;
        }
    }
    sort(begin(a),end(a),[&](PT u,PT v)
    {
        int f1=(u.y>st.y||u.y==st.y&&u.x>st.x)?0:1;
        int f2=(v.y>st.y||v.y==st.y&&v.x>st.x)?0:1;
        if(f1!=f2)return f1<f2;
        int s=sgn(cross(u-st,v-st));
        if(s!=0)return s>0;
        return dot(u-st,u-st)<dot(u-st,v-st);//这里用平方判长度
    });
    vector<PT>ret;

```

```
    for(auto p:a)
    {

while(ret.size()>=2&&is_onRight(p,Line(ret[ret.size()-2],ret.back()))){ret.pop_back
    ()};
        ret.emplace_back(p);
    }
    return ret;
}

T RotateJam(vector<PT>&h)
{
    //h为凸包
    int n=h.size();
    if(n<=2)return 0;//只有2点的情况
    T ans=0;//初始化
    int j=2;
    for(int i=0;i<n;i++)
    {
        PT a=h[i],b=h[(i+1)%n];
        while(sgn(fabs(cross(b-a,h[(j+1)%n]-a))-fabs(cross(b-a,h[j]-a)))>=0)
        {
            j=(j+1)%n;
        }
        //处理答案
        ans;
    }
    return ans;
}
```